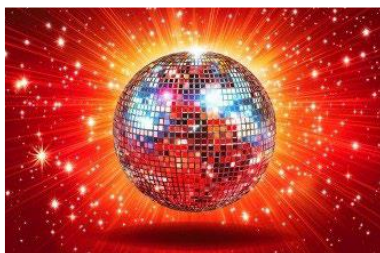


2026 数学建模竞赛 (MCM) 问题 C: 与明星共舞的数据



《与星共舞》(Dancing with the Stars, 简称 DWTS) 是一个国际电视节目系列的美国版本, 原型来自英国节目《Strictly Come Dancing》(最早名为《Come Dancing》)。该节目在阿尔巴尼亚、阿根廷、澳大利亚、中国、法国、印度等许多国家都有版本。美国版本是本题关注的对象, 目前已播出 34 季。

节目中, 明星与专业舞者组成搭档, 每周表演舞蹈。由专业评委组成的评审团对每对选手的表演打分, 同时观众(通过电话或网络)为自己喜欢的选手投票。每位观众每周可以投一次或多次票(有上限限制)。

观众投票的对象是“希望留下的明星”, 而不是直接投票淘汰某人。评委评分与观众投票会结合起来, 决定当周被淘汰的组合(综合得分最低者)。通常有三对(某些赛季更多)进入决赛, 在决赛周中, 根据评委与观众的综合得分对他们进行最终排名(第 1 名到第 3 名, 或第 4、第 5 名)。

评委评分与观众投票的结合方式有多种可能。在美国版的前两季中, 采用的是排名法(rank)。但在第 2 季中, 由于明星选手 Jerry Rice 虽然评委分数很低却进入决赛, 引发争议, 因此节目组改用**百分比法(percent)**来替代排名法。两种方法的示例见附录。

在第 27 季, 又发生了一次争议: 明星 Bobby Bones 尽管评委评分一直较低, 却最终获得冠军。作为回应, 从第 28 季开始, 淘汰机制进行了修改: 先用评委分数与观众投票确定“倒数两名”, 然后在直播节目中由评委投票决定淘汰哪一组。与此同时, 节目制作方也重新采用了最早的“排名法”来结合评委与观众投票。具体从哪一季开始并不完全确定, 但合理推测是第 28 季。

评委评分旨在反映舞者的技术水平, 但舞蹈优劣本身具有一定主观性。观众投票则更具主观性, 不仅受到舞蹈质量影响, 也受到明星人气与个人魅力影响。事实上, 节目制作方可能在一定程度上希望评委与观众之间存在分歧, 因为这种冲突会提升观众兴趣与节目的话题性。

已提供评委评分与选手信息的数据文件。你可以自行补充其他数据，但必须完整注明来源。请基于数据完成以下任务：

问题 1

问题 1 分析（观众投票的估计）

1、开发一个或多个数学模型，估计每位选手在各周获得的观众投票数（真实数据未知且高度保密）。

- 你的模型是否能正确估计出导致真实淘汰结果的观众投票？请给出一致性指标。
- 你对估计结果有多大把握？这种不确定性是否在不同选手/周之间一致？请给出不确定性度量。

这一问的本质是一个典型的“反问题”：真实的观众投票并未给出，但淘汰顺序和评委分数是已知的，因此需要反向推测一种合理的投票分布，使其在官方规则下能够复现真实的比赛结果。由于这是一个信息严重不足的问题，解必然不唯一，因此建模重点不在于寻找“唯一正确答案”，而在于构建一个满足所有淘汰约束、同时尽量符合现实直觉的投票结构。一个自然的思路是将每周的观众投票归一化为比例，在保证被淘汰选手综合得分最低的前提下，寻找最“平滑”的投票分布，例如假设观众行为整体较为分散、不会出现极端集中。这样得到的结果不是一个精确值，而是一组可能解的代表形式，重点是刻画“什么样的投票结构是合理的”。不确定性分析也十分关键，因为不同投票方案都可能解释同一淘汰结果，因此需要通过扰动数据或多次重建来给出投票估计的可信区间，而不是单点结论。

解题思路：

1. 任务重述与核心难点（反问题 + 多解）

赛题要求估计每位选手在每周获得的观众投票（真实值未知且保密），并且要让估计出的投票与评委分数结合后，能够产生与真实淘汰一致的结果，同时还要量化估计的不确定性。由于“投票只通过淘汰结果间接显现”，这是一类典型的反问题：约束不足、解不唯一，所以必须在“所有能解释淘汰的投票方案”中选出一个“最合理”的代表解，并给出可行范围或置信区间来描述不确定性。

2. 数据到数学对象：索引、集合与可观测量

以赛季为单位建模。对每个赛季 s ，定义该赛季出现过的选手集合为 $\mathcal{I}_s$ ，周次集合为 $\mathcal{T}_s = 1, 2, \dots, T_s$ （注意不同赛季周数不同，未播出的周在数据里会出现 N/A）。

对选手 i 、周 t 、评委 k ，数据字段 `weekX_judgeY_score` 对应的分数记为 $x_{s,i,t,k}$ 。当某周不存在第 k 位评委时为 N/A；当选手已被淘汰后，赛题说明其后续周次分数记为 0。

定义当周评委总分（忽略 N/A）为 $J_{s,i,t} = \sum_{k \in \mathcal{K}_{s,t}} x_{s,i,t,k}$ ，其中 $\mathcal{K}_{s,t}$ 是第 t 周实际存在的评委索引集合（通常 3 位，有时 4 位）。

为了识别“当周仍在比赛”的选手集合，定义在赛指示变量 $a_{s,i,t} = \mathbf{1}(J_{s,i,t} > 0)$ ，并令当周

活跃选手集合 $\mathcal{A}_{s,t} = \{i \in \mathcal{I}_s : a_{s,i,t} = 1\}$ 。由于“淘汰后分数记为 0”，用这种方式能从 CSV 直接复原每周剩余人数与参赛者。

进一步，定义第 t 周“本周结束后被淘汰”的集合为 $\mathcal{E}_{s,t} = \{i \in \mathcal{A}_{s,t} : a_{s,i,t+1} = 0\}$ 。若某周无人淘汰则 $\mathcal{E}_{s,t} = \emptyset$ ，若双淘汰则 $|\mathcal{E}_{s,t}| = 2$ ，赛题也明确存在这些情况。

3. 估计目标变量：投票“份额”而非绝对票数

真实票数规模每周可能变动且未知，因此更稳健的做法是估计投票份额（百分比）。令 $v_{s,i,t} \in [0,1]$ 表示赛季 s 第 t 周选手 i 在当周总票数中的份额（fan vote share）。对每周有归一化约束 $\sum_{i \in \mathcal{A}_{s,t}} v_{s,i,t} = 1$ ，以及非负约束 $v_{s,i,t} \geq 0$ 。

若你们需要输出“票数”，可以引入未知的总票量 $V^{\text{tot}}_{s,t}$ 并写为 $\text{Votes}_{s,i,t} = V^{\text{tot}}_{s,t} \cdot v_{s,i,t}$ ，但由于 $V^{\text{tot}}_{s,t}$ 不可识别，论文主体建议以 $v_{s,i,t}$ 为核心输出，必要时可在附录说明如何用任意尺度（如 $V^{\text{tot}}_{s,t} = 10^7$ ）转成“示例票数”。

4. 把节目规则写成约束：百分比法与排名法两套约束体系

赛题给出两种合成方式：早期与推测第 28 季起使用“按排名合成”，第 3 至 27 季使用“按百分比合成”，并给出示例表格说明如何运算。因此我们按赛季选择不同约束。

4.1 百分比法（Season 3–27）：线性约束，最适合大规模反推

百分比法中，评委分数先转为当周百分比。定义 $q_{s,i,t} = \frac{J_{s,i,t}}{\sum_{j \in \mathcal{A}_{s,t}} J_{s,j,t}}$ ，并定义当周合成得分

分 $C_{s,i,t} = q_{s,i,t} + v_{s,i,t}$ 。附录示例正是“评委百分比 + 观众百分比”的相加，然后最小者淘汰。

若当周单淘汰，设真实淘汰者为 e （即 $\mathcal{E}_{s,t} = e$ ），则淘汰一致性约束可写为 $C_{s,e,t} \leq C_{s,j,t}$ 对所有 $j \in \mathcal{A}_{s,t} \setminus e$ 。为避免并列导致的数值不稳定，可加一个极小间隔 $\epsilon > 0$ ，写为 $C_{s,e,t} \leq C_{s,j,t} - \epsilon$ 。在这个体系下，所有约束对变量 $v_{s,i,t}$ 都是线性的，非常适合用线性规划或凸优化一次性求解全赛季。

若当周双淘汰（ $\mathcal{E}_{s,t} = e_1, e_2$ ），可用更保守的两组不等式： $C_{s,e_r,t} \leq C_{s,j,t} - \epsilon$ 对所有 $j \in \mathcal{A}_{s,t} \setminus e_1, e_2$ 且 $r \in 1, 2$ ，表示两位淘汰者的合成得分都不高于任何未淘汰者。

4.2 排名法（Season 1–2 与推测 28+）：用 MILP 或“软排名”处理排序非线性

排名法中，评委分数与观众投票都先转成名次再相加。定义评委名次 $r^J_{s,i,t}$ 为按 $J_{s,i,t}$ 从高到低的排名（名次越小越好），它由数据确定。观众名次 $r^V_{s,i,t}$ 由 $v_{s,i,t}$ 的大小顺序决定（ v 越大名次越小）。合成名次为 $R_{s,i,t} = r^J_{s,i,t} + r^V_{s,i,t}$ ，淘汰者应满足 $R_{s,e,t} \geq R_{s,j,t}$ 对

所有 $j \neq e$ ，这与附录 Season 1 Week 4 的“名次相加”示例一致。

难点是 r^V 是排序函数。若要严格复现，可以做混合整数规划（MILP）：引入二元变量 $y_{s,i,j,t} \in 0,1$ 表示“ i 的投票是否不少于 j ”，即 $y_{s,i,j,t} = 1 \Leftrightarrow v_{s,i,t} \geq v_{s,j,t}$ 。用大 M 线性化比较： $v_{s,i,t} - v_{s,j,t} \geq -M(1 - y_{s,i,j,t})$ 与 $v_{s,i,t} - v_{s,j,t} \leq My_{s,i,j,t}$ 。然后用配对比较累加得到名次，例如可写 $r_{s,i,t}^V = |\mathcal{A}_{s,t}| - \sum_{j \in \mathcal{A}_{s,t}, j \neq i} y_{s,i,j,t}$ （直观解释是“被它不小于的人数越多，名次越靠前”）。最后加入淘汰约束 $r_{s,e,t}^J + r_{s,e,t}^V \geq r_{s,j,t}^J + r_{s,j,t}^V + \epsilon$ 。该方案严格但计算成本较高，适合对关键赛季（如争议赛季）做精确验证。若你们要跑满 34 季且追求效率，可以用“软排名”近似：构造连续的“ i 胜过 j 的概率”

$$P_{i>j} = \frac{1}{1 + \exp\left(\frac{\log v_{s,j,t} - \log v_{s,i,t}}{\tau}\right)}, \text{ 其中 } \tau > 0 \text{ 为温度参数；再定义连续名次}$$

$$\tilde{r}_{s,i,t}^V = 1 + \sum_{j \neq i} (1 - P_{i>j}), \text{ 并用 } \tilde{r}^V \text{ 代替 } r^V \text{ 写淘汰约束 } r_{s,e,t}^J + \tilde{r}_{s,e,t}^V \geq r_{s,j,t}^J + \tilde{r}_{s,j,t}^V + \epsilon。这$$

会变成可微的非线性优化，速度快但属于近似。

5. 多解问题必须配“选择原则”：目标函数（正则化）设计

仅靠淘汰约束通常会得到大量可行解，因此必须提出一个目标函数，在可行域内选出“最合理”的投票方案。两类在竞赛中非常加分且解释性强的选择原则如下。

第一类是最大熵原则，表示在只知道淘汰结果的情况下，不额外引入偏见，让投票分布尽可能“均匀且信息最少”。对每周定义熵 $H_{s,t}(v) = - \sum_{i \in \mathcal{A}_{s,t}} v_{s,i,t} \log v_{s,i,t}$ ，整体目标写为

$$\max \sum_s \sum_{t \in \mathcal{T}_s} H_{s,t}(v), \text{ 并满足上一节的归一化、非负与淘汰约束。这一目标在百分比法下}$$

仍是凸优化（熵为凹函数，最大化凹函数等价凸形式），通常求解稳定。

第二类是跨周平滑，符合“人气不会无缘无故剧烈跳变”的行为假设。可用二次平滑惩罚

$$\min \sum_s \sum_t \sum_{i \in \mathcal{A}_{s,t} \cap \mathcal{A}_{s,t-1}} (v_{s,i,t} - v_{s,i,t-1})^2, \text{ 强调同一选手相邻周投票份额变化尽量小。实战中}$$

$$\text{我们常把两者结合成一个统一目标：} \max \sum_{s,t} H_{s,t}(v) - \lambda \sum_{s,t} (v_{s,i,t} - v_{s,i,t-1})^2, \text{ 其中 } \lambda \geq 0$$

控制“均匀无偏”和“时间连续性”的权衡。这样既能避免极端解，也能避免每周独立求解导致的人气剧烈抖动。

7. 一致性度量：证明估计投票能复现淘汰

题目要求你回答“模型是否能产生与淘汰一致的结果，并给出一致性度量”。最直接的是构造“硬一致率”：对每个存在淘汰的周 t ，定义 $I_{s,t} = 1(\arg \min_{i \in \mathcal{A}_{s,t}} C_{s,i,t} \in \mathcal{E}_{s,t})$ ，总体

$$\text{一致率为 } \text{Consis} = \frac{\sum_s \sum_t I_{s,t}}{\sum_s |\mathcal{T} : \mathcal{E}_{s,t} \neq \emptyset|}。 \text{对双淘汰周可把预测的最低两名与 } \mathcal{E}_{s,t} \text{ 做集合}$$

比较。

同时建议报告“边际（margin）”，反映淘汰结论是否稳健：定义

$m_{s,t} = \min_{j \in A_{s,t} \setminus \mathcal{E}_{s,t}} C_{s,j,t} - \max_{e \in \mathcal{E}_{s,t}} C_{s,e,t}$ 。若 $m_{s,t}$ 很小，说明投票估计刚好卡在边界

上，不确定性会更高；若 $m_{s,t}$ 较大，则淘汰结果对投票扰动不敏感。

8. 不确定性（certainty）建模：两条路线给出“可量化的把握程度”

题面要求你说明估计投票的确定性，并判断这种确定性是否对不同选手/周一致。

这里推荐两条互补路线，论文写起来很有说服力。

第一条是“可行域区间”法：对每个选手-周 (s,i,t) ，在保持所有约束不变的情况下，分别

求 $v^{\min}_{s,i,t} = \min v_{s,i,t}$ 与 $v^{\max}_{s,i,t} = \max v_{s,i,t}$ ，区间宽度

$w_{s,i,t} = v^{\max}_{s,i,t} - v^{\min}_{s,i,t}$ 就是不确定性度量。百分比法下，这两次优化通常仍是线性

/凸问题（只是在目标函数里把某个变量最大化或最小化），计算相对可控；排名法下若用 MILP 会更重，因此可只对少量关键赛季做严格区间，其余用软排名或采样近似。

第二条是“扰动-重建（Bootstrap）”法：考虑评委打分存在主观波动，对每次重采样构造扰动后的总分 $J's,i,t = Js,i,t + \eta_{s,i,t}$ ，其中 $\eta_{s,i,t} \sim \mathcal{N}(0, \sigma^2)$ 或使用更保守的均匀扰动；

每次扰动都重新求解得到 $v^{(b)}_{s,i,t}$ 。然后用样本方差

$\text{Var}(v_{s,i,t}) = \frac{1}{B-1} \sum_b (v^{(b)}_{s,i,t} - \bar{v}_{s,i,t})^2$ 与分位数区间

$\text{CI95\%}(v_{s,i,t}) = [Q_{2.5\%}, Q_{97.5\%}]$ 来表达确定性。你们可以展示“不确定性在临近淘汰边界的选手/周更大”的规律，从而回答“certainty 是否相同”的问题。

9. 你们在论文里可以这样收束“问题一”的结论

最终你们应输出三类结果：第一类是每赛季每周每人的投票份额估计 $\hat{v}_{s,i,t}$ ；第二类是淘

汰一致性指标（比如 Consis 以及每周 margin $m_{s,t}$ 的分布）；第三类是不确定性（比

如区间宽度 $w_{s,i,t}$ 或 Bootstrap 置信区间长度），并解释其随周次、剩余人数、评委分差

大小的变化规律。这样不仅回答“投票是多少”，更回答了“哪些投票估计是稳的、哪些是无法确定的”，这正是评委对反问题最看重的建模质量。

Python 代码：

```
import re
```

```
import math
```

```
from pathlib import Path
```

```

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from scipy.optimize import minimize


# =====
# 1) 读取数据 & 解析列
# =====


def load_data(csv_path: str) -> pd.DataFrame:

    df = pd.read_csv(csv_path)

    return df


def parse_week_judge_columns(df: pd.DataFrame):

    """
    找到所有形如 weekX_judgeY_score 的列，并按 (week, judge) 排序
    """

    pat = re.compile(r"week(\d+)_judge(\d+)_score")

    cols = []

    for c in df.columns:

        m = pat.match(c)

        if m:

            cols.append((int(m.group(1)), int(m.group(2)), c))

    cols.sort(key=lambda x: (x[0], x[1]))

    weeks = sorted({w for w, j, c in cols})

    return cols, weeks

```

```

# =====
# 2) 把某个赛季整理成矩阵
# =====

def season_matrices(df_season: pd.DataFrame, week_judge_cols, max_week: int):
    """
    返回：
    contestants: 该赛季所有选手名字列表（长度 N）
    J: 形状 (T, N) 的评委总分矩阵，T=max_week
    ran: 形状 (T,) 的布尔数组，表示该周节目是否播出（该周所有分数全 NaN 则视为
    未播出）
    """
    contestants = df_season["celebrity_name"].tolist()
    N = len(contestants)
    T = max_week

    J = np.full((T, N), np.nan, dtype=float)
    ran = np.zeros(T, dtype=bool)

    for t in range(1, T + 1):
        cols_t = [c for (w, j, c) in week_judge_cols if w == t]
        mat = df_season[cols_t].to_numpy(dtype=float)

        # 若该周所有选手所有评委分都是 NaN，则认为节目未播出
        if np.all(np.isnan(mat)):
            ran[t - 1] = False
            continue

```

```

        ran[t - 1] = True

        # 评委总分：忽略 NaN

        J[t - 1, :] = np.nansum(mat, axis=1)

    return contestants, J, ran

def active_mask(J_t: np.ndarray):
    """
    在赛判定：分数有限且 > 0（题面说明淘汰后周次记 0）
    """
    return np.isfinite(J_t) & (J_t > 0)

def elimination_set(J: np.ndarray, ran: np.ndarray):
    """
    用“t 周在赛、t+1 周变成 0 => t 周结束后淘汰”识别真实淘汰者集合 E[t]
    """
    T, N = J.shape
    E = [set() for _ in range(T)]
    for t in range(T - 1):
        if (not ran[t]) or (not ran[t + 1]):
            continue

        act_t = active_mask(J[t])
        act_t1 = active_mask(J[t + 1])

        eliminated = np.where(act_t & (~act_t1))[0]

        E[t] = set(eliminated.tolist())

    return E

```



```

# =====
# 3) Percent scheme 反推：最大熵 + 平滑 + 淘汰约束
# =====

def build_problem_percent(
    contestants,
    J: np.ndarray,
    ran: np.ndarray,
    E,
    lam: float = 2.0,
    eps_margin: float = 1e-6
):
    """
    Percent scheme:
    - 评委百分比  $q_{\{t,i\}} = J_{\{t,i\}} / \sum_{\{act\}} J_{\{t,*\}}$ 
    - 合成得分  $C_{\{t,i\}} = q_{\{t,i\}} + v_{\{t,i\}}$ 
    - 真实淘汰者在真实淘汰周满足:  $C_{\{t,e\}} \leq C_{\{t,j\}} - \text{eps}$ 
    变量：只对“该周在赛”的 (t,i) 建立  $v_{\{t,i\}}$ ，并且每周  $\text{sum}(v)=1$ 
    目标：minimize  $\sum v \log v + \text{lam} * \text{smoothness}$ 
           (因为 maximize entropy 等价于 minimize  $\sum v \log v$ )
    """
    T, N = J.shape

    # (t,i) -> 变量下标
    var_index = {}
    idx = 0
    active_sets = {}

```

```

for t in range(T):

    if not ran[t]:

        continue

    act = active_mask(J[t])

    active_sets[t] = np.where(act)[0]

    for i in active_sets[t]:

        var_index[(t, i)] = idx

        idx += 1

nvars = idx

bounds = [(0.0, 1.0)] * nvars

# 计算评委百分比 q

q = {}

for t in range(T):

    if not ran[t]:

        continue

    act = active_sets[t]

    denom = np.nansum(J[t, act])

    if not np.isfinite(denom) or denom <= 0:

        continue

    for i in act:

        q[(t, i)] = J[t, i] / denom

def v_of(x, t, i):

    k = var_index.get((t, i), None)

    return x[k] if k is not None else 0.0

```

```

# 目标函数: minimize sum v log v + lam * sum (v_t - v_{t-1})^2

def objective(x):

    ent_term = 0.0

    for (t, i), k in var_index.items():

        v = x[k]

        ent_term += v * math.log(max(v, 1e-12)) # v log v


    smooth_term = 0.0

    prev_t = None

    for t in range(T):

        if not ran[t]:

            continue

        if prev_t is None:

            prev_t = t

            continue

        common = set(active_sets[prev_t].tolist()).intersection(active_sets[t].tolist())

        for i in common:

            smooth_term += (v_of(x, t, i) - v_of(x, prev_t, i)) ** 2

        prev_t = t


    return ent_term + lam * smooth_term


constraints = []


# (1) 每周归一化 sum v = 1

for t in range(T):

    if not ran[t]:

        continue

    inds = [var_index[(t, i)] for i in active_sets[t] if (t, i) in var_index]

```

```

constraints.append({
    "type": "eq",
    "fun": (lambda inds=inds: (lambda x: np.sum(x[inds]) - 1.0))()
})

# (2) 淘汰约束：真实淘汰者合成得分最低
for t in range(T):
    if (not ran[t]) or (len(E[t]) == 0):
        continue

    act = active_sets[t].tolist()
    eliminated = list(E[t])
    survivors = [j for j in act if j not in E[t]]

    if len(survivors) == 0:
        continue

    for e in eliminated:
        qe = q.get((t, e), 0.0)
        for j in survivors:
            qj = q.get((t, j), 0.0)

            # 要求:  $(qj + vj) - (qe + ve) - \epsilon \geq 0$ 
            constraints.append({
                "type": "ineq",
                "fun": (lambda t=t, e=e, j=j, qe=qe, qj=qj:
                    (lambda x: (qj + v_of(x, t, j)) - (qe + v_of(x, t, e)) -
eps_margin))()
            })

# 初始点：每周均匀分布

```

```

x0 = np.zeros(nvars)

for t in range(T):

    if not ran[t]:

        continue

    act = active_sets[t]

    if len(act) == 0:

        continue

    val = 1.0 / len(act)

    for i in act:

        x0[var_index[(t, i)]] = val

meta = {

    "T": T,

    "N": N,

    "contestants": contestants,

    "J": J,

    "ran": ran,

    "E": E,

    "q": q,

    "active_sets": active_sets,

    "var_index": var_index,

}

return objective, constraints, bounds, x0, meta

```

```

def solve_percent_inverse(meta_builder_output, maxiter=2000, ftol=1e-9):

    objective, constraints, bounds, x0, meta = meta_builder_output

    res = minimize(

        objective,

```

```

        x0,

        method="SLSQP",

        bounds=bounds,

        constraints=constraints,

        options={"maxiter": maxiter, "ftol": ftol, "disp": False},
    )

    if not res.success:

        raise RuntimeError(f"Solve failed: {res.message}")

# unpack to V[t,i]

T, N = meta["T"], meta["N"]

V = np.zeros((T, N), dtype=float)

for (t, i), k in meta["var_index"].items():

    V[t, i] = res.x[k]

return res, meta, V

```

```
# =====
```

```
# 4) 一致性检查：是否复现真实淘汰
```

```
# =====
```

```
def check_consistency_percent(meta, V):
```

```

    T, N = meta["T"], meta["N"]

    ran = meta["ran"]

    E = meta["E"]

    q = meta["q"]

    active_sets = meta["active_sets"]

```

```

preds = {}

elim_weeks = []

for t in range(T):
    if not ran[t]:
        continue
    if len(E[t]) > 0:
        elim_weeks.append(t)

    act = active_sets[t]
    # 合成得分  $C = q + v$ 
    C = {i: (q.get((t, i), 0.0) + V[t, i]) for i in act}
    pred = min(C, key=C.get) if len(C) else None
    preds[t] = pred

correct = 0

for t in elim_weeks:
    if preds.get(t, None) in E[t]:
        correct += 1

rate = correct / len(elim_weeks) if elim_weeks else np.nan

return rate, preds, elim_weeks

```

=====

5) 可视化

=====

```
def plot_heatmap_votes(meta, V, df_season, season_id):
```

```

contestants = meta["contestants"]

ran = meta["ran"]

# 按 placement 排序（更易读）

info = df_season[["celebrity_name",
"placement"]].drop_duplicates().set_index("celebrity_name")

placement = info["placement"].to_dict()

order_names = sorted(contestants, key=lambda n: (placement.get(n, 999), n))

order_idx = [contestants.index(n) for n in order_names]

ran_weeks = [t for t in range(meta["T"]) if ran[t]]

V_plot = V[ran_weeks, :][:, order_idx]

plt.figure(figsize=(12, 6))

plt.imshow(V_plot, aspect="auto")

plt.colorbar(label="Estimated vote share")

plt.yticks(range(len(ran_weeks)), [f"Week {t+1}" for t in ran_weeks])

plt.xticks(range(len(order_names)), order_names, rotation=90)

plt.title(f"Season {season_id}: Estimated fan vote share (Percent-scheme inverse
model)")

plt.tight_layout()

plt.show()

def plot_finalists_trajectories(meta, V, df_season, season_id, top_k=3):

    contestants = meta["contestants"]

    ran = meta["ran"]

    T = meta["T"]

```



```

info = df_season[["celebrity_name",
"placement"]].drop_duplicates().set_index("celebrity_name")

placement = info["placement"].to_dict()

finalists = [n for n in contestants if placement.get(n, 999) <= top_k]
finalists = sorted(finalists, key=lambda n: placement[n])

plt.figure(figsize=(10, 5))

for name in finalists:

    i = contestants.index(name)

    series = [V[t, i] if ran[t] else np.nan for t in range(T)]

    plt.plot(range(1, T + 1), series, marker="o", linewidth=2, label=f'{name}
#{placement[name]}')

plt.xlabel("Week")
plt.ylabel("Estimated vote share")
plt.title(f"Season {season_id}: Vote share trajectories for top {top_k}")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# =====
# 6) 主程序示例：先跑一个赛季（推荐从 27 开始）
# =====

def run_one_season_percent(csv_path: str, season_id: int = 27, lam: float = 2.0):

    df = load_data(csv_path)

```

```

week_judge_cols, weeks = parse_week_judge_columns(df)

max_week = max(weeks)

df_s = df[df["season"] == season_id].copy()

if df_s.empty:
    raise ValueError(f"No data for season {season_id}")

contestants, J, ran = season_matrices(df_s, week_judge_cols, max_week)
E = elimination_set(J, ran)

problem = build_problem_percent(contestants, J, ran, E, lam=lam)
res, meta, V = solve_percent_inverse(problem)

rate, preds, elim_weeks = check_consistency_percent(meta, V)
print(f"[Season {season_id}] Solve success. Hard consistency rate = {rate:.3f}")
print(f"Objective value = {res.fun:.6f}, iterations = {res.nit}")

plot_heatmap_votes(meta, V, df_s, season_id)
plot_finalists_trajectories(meta, V, df_s, season_id, top_k=3)

return meta, V, df_s

if __name__ == "__main__":
    # 改成你的文件路径
    CSV_PATH = "/mnt/data/1f898514-d456-48f2-a79a-8fb5507c58d9.csv"
    run_one_season_percent(CSV_PATH, season_id=27, lam=2.0)

```

问题 2

问题 2 分析（两种投票规则的比较）

2、利用你估计的观众投票数据：

- 比较“排名法”和“百分比法”在各季的结果差异（即对每一季同时应用两种方法）。若结果不同，哪种方法更偏向观众意见？

- 针对存在“争议”的选手（评委与观众分歧较大）进行分析，例如：

- 第 2 季：Jerry Rice（多次评委最低分却获得亚军）
- 第 4 季：Billy Ray Cyrus（6 次评委垫底却获得第 5 名）
- 第 11 季：Bristol Palin（12 次评委最低却获得第 3 名）
- 第 27 季：Bobby Bones（评委分数偏低却夺冠）

不同合成方法是否会导致不同结果？如果再加入“评委在倒数两名中选择淘汰者”的机制，结果会如何变化？

- 基于分析，你会推荐未来赛季使用哪种方法？是否建议保留评委二选一的机制？

这一问的核心并不是简单复现历史，而是进行“反事实分析”：在同一套估计的观众投票条件下，分别应用排名法与百分比法，看结果是否发生变化。也就是说，观众行为被视为固定输入，真正变化的是制度本身。分析重点应放在两种规则对最终排名的敏感程度，例如是否经常导致不同选手被淘汰，是否系统性偏向评委意见或观众意见。对于争议选手，可以通过模拟他们在不同制度下的命运来说明规则差异的实质影响。如果某位选手在一种规则下稳定进入决赛，而在另一种规则下频繁被淘汰，这种现象本身就构成了对制度公平性的有力证据。这一部分的价值不在于算出“谁赢了”，而在于解释“为什么制度会导致这种差异”。

解题思路：

1. 输入与统一符号：把问题 1 的投票估计当作“固定输入”

对赛季 s ，设参赛选手集合为 $\mathcal{I}_s$ ，周次集合为 $\mathcal{T}_s = 1, \dots, T_s$ 。从附件数据可得每周评委总分 $J_{s,i,t}$ （由 `weekX_judgeY_score` 求和得到，忽略 N/A），并可由“淘汰后记 0”识别在赛集合 $\mathcal{A}_{s,t} = \{i \in \mathcal{I}_s : J_{s,i,t} > 0\}$ 以及真实淘汰集合 $\mathcal{E}_{s,t}$ 。

问题 1 已给出观众投票估计（建议用份额），记为 $\hat{v}_{s,i,t}$ ，满足每周归一化 $\sum_{i \in \mathcal{A}_{s,t}} \hat{v}_{s,i,t} = 1$

且 $\hat{v}_{s,i,t} \geq 0$ 。问题 2 中我们将 $\hat{v}_{s,i,t}$ 视为外生固定输入，不再调整；我们只改变合成规则，得到两套“制度输出”。

2. 两种规则的通用计算公式（与附录一致）

2.1 百分比法（Percent scheme）

附录表明百分比法将评委总分转为当周百分比，再与观众投票百分比相加，最低者淘汰。

形式化如下：对当周在赛集合 $\mathcal{A}_{s,t}$ ，定义评委占比 $q_{s,i,t} = \frac{J_{s,i,t}}{\sum_{j \in \mathcal{A}_{s,t}} J_{s,j,t}}$ ，合成得分

$C^{(P)}_{s,i,t} = q_{s,i,t} + \hat{v}_{s,i,t}$ 。则该周预测淘汰者为 $e^{(P)}_{s,t} = \arg \min_{i \in \mathcal{A}_{s,t}} C^{(P)}_{s,i,t}$ ；若当周需要淘汰 k 人（例如双淘汰），则预测淘汰集合为 $E^{(P,k)}_{s,t} = \text{Bottom-}k(C^{(P)}_{s,i,t} | i \in \mathcal{A}_{s,t})$ 。

2.2 排名法（Rank scheme）

附录表明排名法将评委与观众分别排名，再把名次相加，名次和最差者淘汰。

形式化如下：定义评委名次 $r^J s, i, t = \text{RankDesc}(J s, j, t j \in \mathcal{A} s, t)$ ，其中分高者名次小；

定义观众名次 $r^V s, i, t = \text{RankDesc}(\hat{v} s, j, t j \in \mathcal{A} s, t)$ ，票高者名次小；合成名次和为

$R^{(R)} s, i, t = r^J s, i, t + r^V s, i, t$ 。则预测淘汰者为 $e^{(R)} s, t = \arg \max_{i \in \mathcal{A} s, t} R^{(R)} s, i, t$ ；淘汰 k

人时 $E^{(R, k)} s, t = \text{Top-}k(R^{(R)} s, i, t i \in \mathcal{A} s, t)$ 。

3. 反事实模拟框架：用同一份投票估计“跑完整季”

核心是把两种规则都做成一个“周循环淘汰模拟器”。对每个赛季 s ：

1) 初始化在赛集合为 $\mathcal{A}^{(M)} s, 1 = \mathcal{A} s, 1$ ($M \in P, R$ 表示两种机制)。

2) 对周 t ，在 $\mathcal{A}^{(M)} s, t$ 上计算合成指标 ($C^{(P)}$ 或 $R^{(R)}$)，确定当周淘汰集合 $E^{(M, k_t)} s, t$

(k_t 可取真实淘汰人数，也可统一取 1 做标准化比较)。

3) 更新 $\mathcal{A}^{(M)} s, t+1 = \mathcal{A}^{(M)} s, t \setminus E_{s, t}^{(M, k_t)}$ ，进入下一周。

4) 直到决赛周得到最终名次 $\pi_s^{(M)}(i)$ 。

为了让比较“同口径”，建议让 k_t 与真实赛季保持一致： $k_t = |\mathcal{E}_{s, t}|$ (无淘汰周 $k_t = 0$)。

这样两种规则都在“真实赛程结构”下运行，差异来源只来自规则本身。

4. 差异评估指标 (比较什么、怎么量化)

赛题要求比较跨赛季差异，并判断是否“更偏向观众”。建议从三层指标体系展开：周淘汰差异、赛季最终名次差异、偏向性指标。

4.1 周淘汰差异：规则一换，淘汰会不会变？

定义单周差异指示函数 (单淘汰) $D_{s, t} = \mathbf{1}(e^{(P)} s, t \neq e^{(R)} s, t)$ 。若双淘汰则用集合：

$$D_{s, t} = \mathbf{1}(E^{(P, 2)} s, t \neq E^{(R, 2)} s, t)。赛季级差异率定义为 \text{DiffRates} = \frac{\sum_{t \in \mathcal{T}^{\text{elim}} s} D_{s, t}}{|\mathcal{T}_s^{\text{elim}}|},$$

其中 $\mathcal{T}^{\text{elim}} s = \{t : |\mathcal{E} s, t| > 0\}$ 。

为了说明“差异方向是否偏向观众”，可比较两规则淘汰者的观众支持强弱。令

$\Delta v_{s, t} = \hat{v}_{s, e^{(P)} s, t, t} - \hat{v}_{s, e^{(R)} s, t, t}$ ，若 $\Delta v_{s, t} > 0$ ，表示百分比法淘汰者的观众份额更高 (即

百分比法在该周更不“偏观众”)；反之则表示排名法更不“偏观众”。统计 $\Pr(\Delta v_{s, t} > 0)$ 或

其均值即可形成“方向性结论”。

另外可引入“边际差”衡量差异的强弱：在百分比法下定义淘汰边际

$m^{(P)} s, t = \min_{j \notin E^{(P, k_t)} s, t} C^{(P)} s, j, t - \max_{e \in E^{(P, k_t)} s, t} C^{(P)} s, e, t$ ，在排名法下可用名次和的

间隔 $m^{(R)}s, t = \min j \notin E^{(R, k_i)}s, t R^{(R)}s, j, t - \max_{e \in E^{(R, k_i)}s, t} R^{(R)}s, e, t$ 。若 m 很小，说明结果对微小扰动敏感，这类周次是“制度差异更可能出现”的高风险点。

4.2 最终名次差异：冠军/前三会不会变？

定义两规则下的最终名次向量分别为 $\pi_s^{(P)}(i)$ 与 $\pi_s^{(R)}(i)$ （名次越小越好）。最直观指标是冠军一致性： $W_s = \mathbf{1}(\arg \min_i \pi_s^{(P)}(i) = \arg \min_i \pi_s^{(R)}(i))$ 。也可比较前三集合

$T3_s^{(P)} = i : \pi_s^{(P)}(i) \leq 3$ 与 $T3_s^{(R)} = i : \pi_s^{(R)}(i) \leq 3$ ，用 Jaccard 指数

$$\text{Jac}_s = \frac{|T3_s^{(P)} \cap T3_s^{(R)}|}{|T3_s^{(P)} \cup T3_s^{(R)}|}。$$

更系统的做法是名次相关系数：Spearman 相关 $\rho_s = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$ ，其中

$d_i = \pi^{(P)}s(i) - \pi^{(R)}s(i)$ ， n 是比较对象人数（可取进入决赛者或全赛季选手）。也可使用

Kendall 相关 $\tau_s = 1 - \frac{2N\text{discord}}{n(n-1)/2}$ ，其中 $N\text{discord}$ 是两套排序中的不一致对数量。相关

越低，说明规则改变导致的结构性变化越大。

4.3 偏向性指标：哪种规则更贴近观众、哪种更贴近评委？

赛题问“是否某种方法更偏向观众投票”。

一个清晰可解释的框架是：把“观众单方排名”和“评委单方排名”作为参照，然后看制度产生的最终名次更接近哪一个。

在决赛周 t^* （或用整季平均），定义观众单方名次 $\pi^{(V)}s(i) = \text{RankDesc}(\hat{v}s, i, t^*)$ ，评委单方名次 $\pi^{(J)}s(i) = \text{RankDesc}(Js, i, t^*)$ 。然后比较制度名次与单方名次的相关性，例如

$\rho_s^{(P,V)} = \text{Spearman}(\pi_s^{(P)}, \pi_s^{(V)})$ 与 $\rho_s^{(P,J)} = \text{Spearman}(\pi_s^{(P)}, \pi_s^{(J)})$ ，同理得到 $\rho_s^{(R,V)}$ 、

$\rho_s^{(R,J)}$ 。如果某规则普遍满足 $\rho_s^{(P,V)} > \rho_s^{(P,J)}$ ，即可解释为“更偏向观众”；反之更偏向评委。

更“机制化”的偏向度量可以用“可调权重”做灵敏度分析：定义连续混合得分 $C^{(\alpha)}s, i, t = \alpha qs, i, t + (1 - \alpha)\hat{v}_{s, i, t}$ ，其中 $\alpha \in [0, 1]$ 。当 $\alpha = 0$ 完全由观众决定，当

$\alpha = 1$ 完全由评委决定。对于每周可以计算“淘汰者切换临界点” $\alpha_{s, t}^*$ ：例如比较两个候

选淘汰者 a 与 b 的得分差 $\Delta C^{(\alpha)}a, b = (q_a - q_b)\alpha + (\hat{v}_a - \hat{v}_b)(1 - \alpha)$ ，令

$\Delta C^{(\alpha)}a, b = 0$ 可解得 $\alpha^* = \frac{\hat{v}_b - \hat{v}_a}{(q_a - q_b) + (\hat{v}_b - \hat{v}_a)}$ （需要分母非零）。统计 $\alpha_{s, t}^*$ 的分布可

以回答：在多大范围内“更偏观众”不会改变淘汰、在多大范围内“稍微偏评委”就会改变淘汰，从而比较两种离散规则在连续谱上的位置与敏感性。

5. 争议选手的专项反事实：只换规则，看命运是否改变

赛题列出争议案例（如第 2 季 Jerry Rice、第 4 季 Billy Ray Cyrus、第 11 季 Bristol Palin、第 27 季 Bobby Bones 等），要求你判断“若换合成规则是否仍会得到相同结果”。这里建议做“路径比较”而不是只看决赛周：

对某争议选手 i^* ，记录其在两规则下每周的风险值，例如百分比法风险

$R^{\text{risk}}_t(i^*) = \text{RankAsc}(C^{(P)}_s, i^*, t)$ （越靠近 1 越危险，表示越接近淘汰），排名法风险可

用 $\text{RankDesc}(R^{(R)}_s, i^*, t)$ 。然后比较其“最危险周”是否发生变化、是否在某规则下提前

被淘汰。定义选手的“制度生存差” $\Delta T(i^*) = T^{(P)}_{\text{survive}}(i^*) - T^{(R)}_{\text{survive}}(i^*)$ ，其中

$T^{(M)}_{\text{survive}}$ 是在规则 M 下存活到的周数（或最终名次）。若 ΔT 明显非零，就可形

成强结论：“仅改变合成规则，就足以改变该争议选手的命运”。

6. 加入“倒数两名评委二选一”机制：如何建模其影响（Season 28+ 讨论点）

赛题说明从第 28 季左右引入机制：先用评委分数与观众投票确定倒数两名，再由评委在直播中投票淘汰其中一人。

对建模而言，可以把这一机制分成两步：

第一步：用某种合成规则（可分别用百分比法或排名法）确定倒数两名集合

$B^{(M)}_s, t = \text{Bottom2}(\text{score under } M)$ ，其中 $M \in P, R$ 。例如在百分比法下

$B^{(P)}_s, t = \text{Bottom2}(C^{(P)}_s, i, t)$ ，排名法下 $B^{(R)}_s, t = \text{Top2}(R^{(R)}_{s, i, t})$ （因为名次和越大越差）。

第二步：评委在倒数两名中选择淘汰者。由于缺少评委投票细节，可做两种合理模型：

（A）确定性模型：评委淘汰“评委分更低者”，即若 $B = a, b$ 且 $J_{s, a, t} < J_{s, b, t}$ ，则淘汰 a ；

（B）概率模型：评委更偏向淘汰评委分低者，但允许随机性，可设

$$P(\text{eliminate } a) = \frac{\exp(\gamma(J_{s, b, t} - J_{s, a, t}))}{1 + \exp(\gamma(J_{s, b, t} - J_{s, a, t}))}, \text{ 其中 } \gamma > 0 \text{ 表示评委一致性强度。}$$

用该二阶段机制，你可以重新跑完整季得到 $\pi_s^{(M, \text{judge-save})}$ ，并与不带二阶段机制的 $\pi_s^{(M)}$

做差异分析，回答“加入评委二选一会不会压制观众争议、减少爆冷”。

7. 最终报告结构建议：把“比较”写成可评分的结论链

你们在论文中建议按以下逻辑输出：

（1）固定 $\hat{v}_{s, i, t}$ ，分别计算 $C^{(P)}_s, i, t$ 与 $R^{(R)}_s, i, t$ ，给出全赛季淘汰差异率 DiffRate_s 的分布（跨 34 季）。

(2) 给出冠军一致性 W_s 、前三集合 Jaccard Jac_s 、以及名次相关 ρ_s 或 τ_s ，说明“制度改变对最终名次影响程度”。

(3) 用 $\rho_s^{(\cdot,V)}$ 与 $\rho^{(\cdot,J)}_s$ 的对比来回答“哪种更偏向观众”，并辅以 α 灵敏度或临界点 $\alpha^*_{s,t}$ 的解释，提升机制洞察。

(4) 对争议选手给出路径级反事实（例如生存周数差 ΔT 、每周风险排名变化），再加上“倒数两名评委二选一”的二阶段模拟，形成对节目制作方的建议闭环

Python 代码:

```
import re
import math
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# =====
# 0) 读取数据 & 解析 week/judge 列
# =====

def load_data(csv_path: str) -> pd.DataFrame:
    return pd.read_csv(csv_path)

def parse_week_judge_columns(df: pd.DataFrame):
    pat = re.compile(r"week(\d+)_judge(\d+)_score")
    cols = []
    for c in df.columns:
        m = pat.match(c)
```

```

        if m:

            cols.append((int(m.group(1)), int(m.group(2)), c))

    cols.sort(key=lambda x: (x[0], x[1]))

    weeks = sorted({w for w, j, c in cols})

    return cols, weeks

# =====

# 1) 赛季矩阵化: J、ran、在赛集合、真实淘汰集合

# =====

def season_matrices(df_season: pd.DataFrame, week_judge_cols, max_week: int):

    contestants = df_season["celebrity_name"].tolist()

    N = len(contestants)

    T = max_week

    J = np.full((T, N), np.nan, dtype=float)

    ran = np.zeros(T, dtype=bool)

    for t in range(1, T + 1):

        cols_t = [c for (w, j, c) in week_judge_cols if w == t]

        mat = df_season[cols_t].to_numpy(dtype=float)

        if np.all(np.isnan(mat)):

            ran[t - 1] = False

            continue

        ran[t - 1] = True

        J[t - 1, :] = np.nansum(mat, axis=1)

```



```

    return contestants, J, ran

def active_mask(J_t: np.ndarray):
    return np.isfinite(J_t) & (J_t > 0)

def elimination_set(J: np.ndarray, ran: np.ndarray):
    """
    用: t 周在赛、t+1 周变成 0 => t 周结束后淘汰
    返回 E[t] 为一个 set, 里面是被淘汰者的 index (0-based)
    """
    T, N = J.shape
    E = [set() for _ in range(T)]
    for t in range(T - 1):
        if (not ran[t]) or (not ran[t + 1]):
            continue

        act_t = active_mask(J[t])
        act_t1 = active_mask(J[t + 1])
        eliminated = np.where(act_t & (~act_t1))[0]
        E[t] = set(eliminated.tolist())

    return E

def elimination_counts(E):
    """k_t = 每周真实淘汰人数 (用于规则模拟的同口径淘汰人数) """
    return [len(e) for e in E]

# =====
# 2) 问题 1 (简化版): Percent scheme 反推投票份额  $\hat{v}$ 

```

```

# 目标：最大熵 + 平滑；约束：sum(v)=1、淘汰一致
# =====

def build_problem_percent_inverse(contestants, J, ran, E, lam=2.0, eps_margin=1e-6):
    T, N = J.shape

    # 只对“该周在赛”的 (t,i) 建立变量
    var_index = {}
    active_sets = {}
    idx = 0

    for t in range(T):
        if not ran[t]:
            continue

        act = np.where(active_mask(J[t]))[0]
        active_sets[t] = act

        for i in act:
            var_index[(t, i)] = idx
            idx += 1

    nvars = idx
    bounds = [(0.0, 1.0)] * nvars

    # 评委百分比 q
    q = {}

    for t in range(T):
        if not ran[t]:
            continue

        act = active_sets[t]
        denom = np.nansum(J[t, act])

        if not np.isfinite(denom) or denom <= 0:

```

```

        continue

    for i in act:

         $q[(t, i)] = J[t, i] / \text{denom}$ 

def v_of(x, t, i):

    k = var_index.get((t, i), None)

    return x[k] if k is not None else 0.0

def objective(x):

    # minimize sum v log v + lam * smoothness

    ent = 0.0

    for (t, i), k in var_index.items():

        v = x[k]

        ent += v * math.log(max(v, 1e-12))

    smooth = 0.0

    prev_t = None

    for t in range(T):

        if not ran[t]:

            continue

        if prev_t is None:

            prev_t = t

            continue

        common = set(active_sets[prev_t].tolist()).intersection(active_sets[t].tolist())

        for i in common:

            smooth += (v_of(x, t, i) - v_of(x, prev_t, i))**2

        prev_t = t

    return ent + lam * smooth

```

```

constraints = []

# 每周归一化 sum v = 1
for t in range(T):
    if not ran[t]:
        continue

    inds = [var_index[(t, i)] for i in active_sets[t]]
    constraints.append({
        "type": "eq",
        "fun": (lambda inds=inds: (lambda x: np.sum(x[inds]) - 1.0))()
    })

# 淘汰约束 (Percent): 真实淘汰者合成得分最低
for t in range(T):
    if not ran[t]:
        continue

    if len(E[t]) == 0:
        continue

    act = active_sets[t].tolist()
    eliminated = list(E[t])
    survivors = [j for j in act if j not in E[t]]

    for e in eliminated:
        qe = q.get((t, e), 0.0)

        for j in survivors:
            qj = q.get((t, j), 0.0)
            constraints.append({
                "type": "ineq",
                "fun": (lambda t=t, e=e, j=j, qe=qe, qj=qj:
                    (lambda x: (qj + v_of(x, t, j)) - (qe + v_of(x, t, e)) -

```

```

eps_margin))()

        })

# 初始点：每周均匀
x0 = np.zeros(nvars)
for t in range(T):
    if not ran[t]:
        continue

    act = active_sets[t]
    val = 1.0 / len(act) if len(act) > 0 else 0.0
    for i in act:
        x0[var_index[(t, i)]] = val

meta = {
    "T": T, "N": N,
    "contestants": contestants,
    "J": J, "ran": ran,
    "E": E, "q": q,
    "active_sets": active_sets,
    "var_index": var_index
}

return objective, constraints, bounds, x0, meta

def solve_percent_inverse(contestants, J, ran, E, lam=2.0):
    objective, constraints, bounds, x0, meta = build_problem_percent_inverse(
        contestants, J, ran, E, lam=lam
    )
    res = minimize(
        objective,

```

```

        x0,

        method="SLSQP",

        bounds=bounds,

        constraints=constraints,

        options={"maxiter": 2000, "ftol": 1e-9, "disp": False}
    )

    if not res.success:

        raise RuntimeError(f"Vote estimation failed: {res.message}")

    T, N = meta["T"], meta["N"]

    V = np.zeros((T, N), dtype=float)

    for (t, i), k in meta["var_index"].items():

        V[t, i] = res.x[k]

    return V, meta, res

```

=====

3) 问题 2：两种规则的“逐周淘汰模拟”

=====

```

def rank_desc(values: np.ndarray):
    """
    返回名次（1 最好）：值越大名次越小
    并列处理：平均名次（dense/competition 都可，这里用平均名次）
    """
    # argsort descending
    order = np.argsort(-values)
    ranks = np.empty_like(values, dtype=float)

```

```
# values with ties
```

```
sorted_vals = values[order]
```

```
n = len(values)
```

```
i = 0
```

```
while i < n:
```

```
    j = i
```

```
    while j + 1 < n and sorted_vals[j + 1] == sorted_vals[i]:
```

```
        j += 1
```

```
    # i..j are tied; average rank = (i+1 + j+1)/2
```

```
    avg_rank = (i + 1 + j + 1) / 2.0
```

```
    ranks[order[i:j+1]] = avg_rank
```

```
    i = j + 1
```

```
return ranks
```

```
def percent_scores(J_t: np.ndarray, V_t: np.ndarray, active_idx: np.ndarray):
```

```
    """
```

```
     $C^{\{P\}} = q + v$ 
```

```
    """
```

```
    J_act = J_t[active_idx]
```

```
    denom = np.sum(J_act)
```

```
    q = J_act / denom if denom > 0 else np.zeros_like(J_act)
```

```
    C = q + V_t[active_idx]
```

```
    return C
```

```
def rank_scores(J_t: np.ndarray, V_t: np.ndarray, active_idx: np.ndarray):
```

```
    """
```

```
     $R^{\{R\}} = r^J + r^V$ 
```

```
    """
```

```

J_act = J_t[active_idx]

V_act = V_t[active_idx]

rJ = rank_desc(J_act)

rV = rank_desc(V_act)

R = rJ + rV

return R

```

```
def simulate_season(J, ran, V_hat, k_list, rule="percent"):
```

```
    """
```

在固定 V_hat 下，按 $rule$ 模拟整季淘汰路径。

$k_list[t]$ = 第 t 周真实淘汰人数（0/1/2...），用于同口径比较。

返回：

elim_path: 每周淘汰者 index 列表（list of list）

final_order: 最后剩下的选手 index（从“冠军”到“最后一名”的排序）

```
    """
```

```
T, N = J.shape
```

```
alive = set(np.where(active_mask(J[np.where(ran)[0][0]]))[0].tolist()) # 初始在赛
```

```
elim_path = []
```

```
for t in range(T):
```

```
    if not ran[t]:
```

```
        elim_path.append([])
```

```
        continue
```

```
    k = k_list[t]
```

```
    if k == 0:
```

```
        elim_path.append([])
```

```
        continue
```

```
    active_idx = np.array(sorted(list(alive)), dtype=int)
```



```

if rule == "percent":
    score = percent_scores(J[t], V_hat[t], active_idx)
    # score 越小越差 -> bottom k
    bad_local = np.argsort(score)[:k]
elif rule == "rank":
    score = rank_scores(J[t], V_hat[t], active_idx)
    # R 越大越差 -> top k
    bad_local = np.argsort(-score)[:k]
else:
    raise ValueError("rule must be 'percent' or 'rank'")

eliminated = active_idx[bad_local].tolist()

for e in eliminated:
    if e in alive:
        alive.remove(e)

elim_path.append(eliminated)

# final_order: 最后 alive 里的作为前几名；为了排序，用最后一周的规则得分排序
alive_idx = np.array(sorted(list(alive)), dtype=int)
last_week = max([t for t in range(T) if ran[t]])
if len(alive_idx) > 1:
    if rule == "percent":
        score = percent_scores(J[last_week], V_hat[last_week], alive_idx)
        final_order = alive_idx[np.argsort(-score)].tolist() # score 大更好
    else:
        score = rank_scores(J[last_week], V_hat[last_week], alive_idx)
        final_order = alive_idx[np.argsort(score)].tolist() # R 小更好
else:

```

```

        final_order = alive_idx.tolist()

    return elim_path, final_order

# =====
# 4) 指标计算：DiffRate、冠军是否变、名次相关
# =====

def diff_rate(elimA, elimB, ran, k_list):
    """
    按淘汰周比较集合是否相同（同 k_t 口径）
    """
    T = len(elimA)
    diffs = []
    for t in range(T):
        if (not ran[t]) or (k_list[t] == 0):
            continue
        diffs.append(1 if set(elimA[t]) != set(elimB[t]) else 0)
    return float(np.mean(diffs)) if diffs else np.nan

def spearman_rank_corr(rank1: dict, rank2: dict):
    """
    rank1, rank2: {i: rank}, rank 越小越好
    Spearman 相关（对名次）
    """
    keys = sorted(set(rank1.keys()).intersection(rank2.keys()))
    if len(keys) < 3:
        return np.nan

```

```

x = np.array([rank1[k] for k in keys], dtype=float)

y = np.array([rank2[k] for k in keys], dtype=float)

# 相关系数

x = x - x.mean()

y = y - y.mean()

denom = np.sqrt(np.sum(x*x) * np.sum(y*y))

return float(np.sum(x*y) / denom) if denom > 0 else np.nan


def order_to_ranks(order: list):
    """
    order: [winner, second, third,...]

    -> ranks dict: {i: 1, 2, 3,...}

    """
    return {i: (idx + 1) for idx, i in enumerate(order)}


# =====

# 5) 可视化: 跨赛季 DiffRate 柱状图、单赛季路径对比

# =====


def plot_diff_rates(summary_df: pd.DataFrame):
    plt.figure(figsize=(12, 5))

    plt.bar(summary_df["season"].astype(str), summary_df["DiffRate"])

    plt.xticks(rotation=90)

    plt.ylabel("DiffRate (weekly elimination set differs)")

    plt.title("Percent vs Rank: Weekly elimination difference rate by season")

    plt.grid(True, axis="y", alpha=0.3)

    plt.tight_layout()

    plt.show()

```

```

def plot_champion_changes(summary_df: pd.DataFrame):

    plt.figure(figsize=(10, 4))

    plt.bar(summary_df["season"].astype(str), summary_df["ChampionChanged"].astype(int))

    plt.xticks(rotation=90)

    plt.ylabel("Champion changed? (1=yes)")

    plt.title("Percent vs Rank: Champion changed by season")

    plt.grid(True, axis="y", alpha=0.3)

    plt.tight_layout()

    plt.show()


def plot_season_elim_comparison(contestants, elimP, elimR, ran, k_list, season_id):

    """

    画一个“周-规则”的淘汰对比图：每周显示两规则淘汰者姓名

    """

    weeks = []

    P_names = []

    R_names = []

    for t in range(len(elimP)):

        if (not ran[t]) or (k_list[t] == 0):

            continue

        weeks.append(t + 1)

        P_names.append(", ".join([contestants[i] for i in elimP[t]]))

        R_names.append(", ".join([contestants[i] for i in elimR[t]]))

    # 文本表更清晰：用 matplotlib 画表

    fig, ax = plt.subplots(figsize=(12, max(4, 0.35 * len(weeks))))

    ax.axis("off")

    table_data = [[w, pn, rn, int(pn != rn)] for w, pn, rn in zip(weeks, P_names, R_names)]

```

```

col_labels = ["Week", "Percent eliminated", "Rank eliminated", "Different?"]

table = ax.table(cellText=table_data, colLabels=col_labels, loc="center")

table.auto_set_font_size(False)

table.set_fontsize(9)

table.scale(1, 1.2)

plt.title(f"Season {season_id}: Elimination comparison (Percent vs Rank)")

plt.tight_layout()

plt.show()

# =====

# 6) 主程序：选择赛季跑（推荐先跑争议赛季）

# =====

def run_problem2(
    csv_path: str,
    seasons_to_run=None,
    lam_for_vote_est=2.0,
    estimate_votes=True
):
    """
    seasons_to_run:
        - None: 默认只跑 season 27（快速验证）
        - list: 例如 [2,4,11,27] 或 list(range(3,28))

    estimate_votes:
        - True: 先用问题 1 反推 v_hat（Percent inverse）
        - False: 需要你自己提供 v_hat（例如你已缓存）
    """

    df = load_data(csv_path)

```

```

week_judge_cols, weeks = parse_week_judge_columns(df)

max_week = max(weeks)

if seasons_to_run is None:
    seasons_to_run = [27]

summary_rows = []

for s in seasons_to_run:
    df_s = df[df["season"] == s].copy()
    if df_s.empty:
        continue

    contestants, J, ran = season_matrices(df_s, week_judge_cols, max_week)
    E_true = elimination_set(J, ran)
    k_list = elimination_counts(E_true)

    # --- 估计投票份额 v_hat (问题 1) ---
    if estimate_votes:
        V_hat, meta, res = solve_percent_inverse(contestants, J, ran, E_true,
lam=lam_for_vote_est)
    else:
        raise ValueError("estimate_votes=False requires you to provide V_hat
externally.")

    # --- 两种规则模拟 ---
    elimP, finalP = simulate_season(J, ran, V_hat, k_list, rule="percent")
    elimR, finalR = simulate_season(J, ran, V_hat, k_list, rule="rank")

```

```

# --- 指标 ---

dr = diff_rate(elimP, elimR, ran, k_list)

ranksP = order_to_ranks(finalP)

ranksR = order_to_ranks(finalR)


champP = finalP[0] if len(finalP) > 0 else None

champR = finalR[0] if len(finalR) > 0 else None

champ_changed = (champP != champR) if (champP is not None and champR is not
None) else np.nan


rho = spearman_rank_corr(ranksP, ranksR)


summary_rows.append({

    "season": s,

    "n_contestants": len(contestants),

    "DiffRate": dr,

    "ChampionChanged": bool(champ_changed) if isinstance(champ_changed, bool)
else champ_changed,

    "Spearman_FinalRanks": rho,

    "Champion_P": contestants[champP] if champP is not None else None,

    "Champion_R": contestants[champR] if champR is not None else None

})


# --- 单赛季可视化（默认只对 27 或少量赛季画） ---

if len(seasons_to_run) <= 5:

    plot_season_elim_comparison(contestants, elimP, elimR, ran, k_list, s)


summary_df =
pd.DataFrame(summary_rows).sort_values("season").reset_index(drop=True)

```

```

# 跨赛季可视化

if len(summary_df) >= 2:

    plot_diff_rates(summary_df)

    plot_champion_changes(summary_df)

return summary_df

if __name__ == "__main__":

    CSV_PATH = "/mnt/data/1f898514-d456-48f2-a79a-8fb5507c58d9.csv"

    # 推荐先跑争议赛季：2,4,11,27（如果跑太慢就先跑 [27]）

    summary = run_problem2(

        CSV_PATH,

        seasons_to_run=[27],      # 改成 [2,4,11,27] 或 list(range(3,28))

        lam_for_vote_est=2.0,

        estimate_votes=True

    )

    print(summary)

```

问题 3

问题 3 分析（明星与舞者特征的影响）

3、利用数据（包括你估计的观众投票），建立模型分析专业舞者和明星特征（如年龄、行业等）对比赛结果的影响。这些因素对评委评分与观众投票的影响是否一致？

这一问本质上是一个因果与关联分析问题，目标是探究哪些个体特征会影响比赛结果，以及这些因素对评委和观众是否产生不同作用。建模时应将评委分数和观众投票视为两个不同的结果变量，分别分析它们与明星年龄、行业背景、国籍以及专业舞者身份之间的关系。如果某些特征对评委分数影响显著，但对观众投票影响很小，说明评委更偏重技术属性；反之，如果某些特征只对观众有效，则反映的是人气或娱乐效应。这一对比能够揭示比赛中“专业评价体系”和“大众审美体系”之间的结构性差异，是整道题中最贴近现实社会解释的一部分。

解题思路：

1. 数据与变量：从附件字段构造“特征—响应”结构

对每个赛季 s 、选手 i 、周次 t ，可从附件直接得到明星特征：行业（类别） $I_{s,i}$ （celebrity_industry）、年龄 $A_{s,i}$ （celebrity_age_during_season）、所在州/国家 $H_{s,i}$ （celebrity_homestate、celebrity_homecountry/region），以及其搭档专业舞者身份 $P_{s,i}$ （ballroom_partner，为类别变量）。

评委方面，从 weekX_judgeY_score 可计算当周评委总分 $J_{s,i,t} = \sum_{k \in K_{s,t}} x_{s,i,t,k}$ ，并用“淘汰后记 0”判定在赛集合 $As,t = i : Js,i,t > 0$ 。

观众投票方面，真实票数未知，因此使用问题 1 估计的投票份额 $\hat{v}_{s,i,t}$ （每周归一化

$\sum_{i \in As,t} \hat{v}_{s,i,t} = 1$ ）。这使得问题 3 的关键响应变量（观众偏好）可被纳入统计建模。

2. 关键建模策略：同一批特征，对两类响应分别建模

为了回答“影响是否一致”，必须建立两套模型：

- 评委模型：解释 $J_{s,i,t}$ 或其标准化形式；
- 观众模型：解释 $\hat{v}_{s,i,t}$ 或其变换形式。

为避免赛季与周次规模差异导致混淆，建议使用“标准化响应”，例如对每个赛季每周做相对化：

评委百分比（当周相对评分）定义为 $q_{s,i,t} = \frac{J_{s,i,t}}{\sum_{j \in As,t} J_{s,j,t}}$ ，满足 $\sum_{i \in As,t} q_{s,i,t} = 1$ 。观众投票

票份额已是归一化的 $\hat{v}_{s,i,t}$ 。这样两者同尺度，便于比较。

3. 特征工程：把“明星/舞者特征”转成可回归的设计矩阵

令每个样本对应一个三元组 (s,i,t) （仅取在赛的周次）。构造特征向量 $x_{s,i,t}$ ：

- 1) 数值特征：年龄 $A_{s,i}$ ；也可加入 $A_{s,i}^2$ 捕捉非线性，即 $[A_{s,i}, A_{s,i}^2]$ 。
- 2) 类别特征：行业 $I_{s,i}$ 、州/国家 $H_{s,i}$ 、专业舞者 $P_{s,i}$ ，做 one-hot 编码。
- 3) 赛季/周次特征：赛季固定效应 δ_s 、周次固定效应 γ_t （或只在赛季内使用周次效应）。
- 4) 动态表现特征（可选但很加分）：前几周平均评委分 $\bar{J}^{(3)}_{s,i} = \frac{1}{3} \sum_{t=1}^3 J_{s,i,t}$ 、以及投票增长率 $g_{s,i} = \text{slope}(\hat{v}_{s,i,t})$ ，用于刻画“初始实力/人气”和“人气成长”。

因此可写成 $x_{s,i,t} = [1, A_{s,i}, A_{s,i}^2, \text{OneHot}(I_{s,i}), \text{OneHot}(H_{s,i}), \text{OneHot}(P_{s,i}), \delta_s, \gamma_t, \dots]$ 。

4. 模型 A：评委评分通道（Judge channel）

4.1 基础线性混合效应模型（推荐主模型）

用 $q_{s,i,t}$ 作为响应变量，构造：

$$q_{s,i,t} = x_{s,i,t}^\top \beta^{(J)} + u^{(J)} Ps,i + u^{(J)} s + \epsilon^{(J)} s,i,t。$$

其中 $\beta^{(J)}$ 描述明星特征对评委评分的影响； $u^{(J)} Ps,i$ 是专业舞者的随机效应（不同舞者可能整体更强）； $u^{(J)} s$ 是赛季随机效应（赛季整体打分尺度变化）；误差项满足 $\epsilon^{(J)} s,i,t \sim \mathcal{N}(0, \sigma_J^2)$ 。

如果不做随机效应（实现更简单），可用固定效应回归：

$$q_{s,i,t} = x_{s,i,t}^\top \beta^{(J)} + \epsilon_{s,i,t}^{(J)},$$

并用赛季 dummy 与舞者 dummy 作为固定效应加入 x 。

4.2 稳健性：对“评委人数变化/奖励分”进行处理

题面指出不同赛季评委人数不同、存在奖励分、团队舞取平均等。采用 $q_{s,i,t}$ （占比）本身能缓解“绝对分数尺度”差异；同时可用 winsorize 或 Huber 损失做稳健回归，以减少奖励分带来的异常影响。

5. 模型 B：观众投票通道（Fan channel）

观众投票份额 $\hat{v}_{s,i,t}$ 在 $(0,1)$ 内并且每周总和为 1，直接线性回归可能违背边界。常见两种处理：

5.1 Logit 变换回归（简单可用）

定义 $y^{(V)} s,i,t = \log\left(\frac{\hat{v} s,i,t}{1-\hat{v} s,i,t}\right)$ ，然后做线性/混合效应模型：

$$y^{(V)} s,i,t = x s,i,t^\top \beta^{(V)} + u^{(V)} Ps,i + u^{(V)} s + \epsilon^{(V)} s,i,t，$$

其中 $\epsilon_{s,i,t}^{(V)} \sim \mathcal{N}(0, \sigma_V^2)$ 。

为了避免 $\hat{v}$ 为 0 或 1（淘汰后周为 0），应仅对在赛周次建模，并做截断 $\hat{v} \leftarrow \min(\max(\hat{v}, \varepsilon), 1-\varepsilon)$ 。

5.2 Dirichlet 回归（更机制一致、冲奖更强）

由于每周投票份额向量 $\hat{v}_{s,:t}$ 满足 simplex 约束，可认为其服从 Dirichlet 分布：

$$(\hat{v}_{s,1,t}, \dots, \hat{v}_{s,n_t,t}) \sim \text{Dirichlet}(\alpha_{s,1,t}, \dots, \alpha_{s,n_t,t}),$$

并令参数与特征相关： $\alpha_{s,i,t} = \exp(x_{s,i,t}^\top \beta^{(V)})$ 。则可用极大似然估计 $\beta^{(V)}$ ，并且天然符合“每周总和为 1”的结构。这一模型写在论文里很加分（属于匹配数据结构的统计模型）。

6. “影响是否一致”的核心比较：系数差异与重要性差异

赛题要求判断明星/舞者特征对评委与观众的影响是否一致。你们可以用以下两类量化方法（建议都做，互为佐证）。

6.1 系数对比（解释性最强）

若两套模型得到系数 $\beta^{(J)}$ 与 $\beta^{(V)}$ ，对每个特征维度 k 定义差异：

$$\Delta_k = \beta_k^{(V)} - \beta_k^{(J)}。$$

若 $\Delta_k > 0$ 且显著，则表示该特征更受观众偏爱；若 $\Delta_k < 0$ 则该特征更受评委认可。

显著性可用 bootstrap 或标准误构造 z 值：
$$z_k = \frac{\Delta_k}{\sqrt{\text{SE}(\beta_k^{(V)})^2 + \text{SE}(\beta_k^{(J)})^2}}。$$

此外可以画“系数散点图”：横轴 $\beta_k^{(J)}$ ，纵轴 $\beta_k^{(V)}$ ，若点集中在对角线附近说明影响一致；若偏离对角线说明“评委与观众偏好割裂”。

6.2 变量重要性对比（更偏预测/解释折中）

如果你们用随机森林、XGBoost 或 Lasso 回归做替代模型，可对两通道分别计算重要性 $\text{Imp}^{(J)}(k)$ 与 $\text{Imp}^{(V)}(k)$ ，然后比较比值：

$$R_k = \frac{\text{Imp}^{(V)}(k)}{\text{Imp}^{(J)}(k)}。$$

R_k 大说明该特征更驱动观众投票； R_k 小说明更驱动评委评分。这样即使存在非线性和交互也能捕捉。

7. 将“专业舞者”影响单独提炼（题目重点之一）

题面明确提到“专业舞者和明星特征对比赛结果的影响”。专业舞者的影响可以通过随机效应或固定效应估计：

若采用随机效应模型，可得到每个舞者的效应估计 $\hat{u}_p^{(J)}$ 与 $\hat{u}_p^{(V)}$ 。然后定义舞者的“评委优势”和“观众优势”：

$$S_p = \hat{u}_p^{(J)}, \quad F_p = \hat{u}_p^{(V)},$$

并计算相关性 $\rho_{S,F} = \frac{\text{cov}(S,F)}{\sigma_S \sigma_F}$ 。若 $\rho_{S,F}$ 高，说明强舞者既能提高技术评分也能提高

人气；若低或为负，说明存在“技术型舞者”和“娱乐型舞者”的分化。

8. 结果解释输出：落到“比赛结果”层面

除了分析 q 与 $\hat{v}$ ，还可以把最终名次作为结果变量，做一个“结构方程”式的解释：

首先定义综合表现变量 $Z_{s,i,t} = w_J q_{s,i,t} + w_V \hat{v}_{s,i,t}$ （权重可取 0.5/0.5 或按赛制），然后定义存活风险或淘汰概率，例如用离散时间生存模型：

$$P(\text{elim at } t | \text{ alive to } t) = \sigma(\theta_0 + \theta_1 Z_{s,i,t} + \theta^\top x_{s,i,t}),$$

其中 $\sigma(x) = \frac{1}{1 + \exp(-x)}$ 。这样可以把“特征影响”最终解释到“更容易被淘汰还是更容易

晋级”，增强现实意义。

Python 代码:

```
import re
import math
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import minimize

from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import r2_score, mean_squared_error

# =====
# 1) 读入数据 & 解析列
# =====

CSV_PATH = "/mnt/data/1f898514-d456-48f2-a79a-8fb5507c58d9.csv"
df = pd.read_csv(CSV_PATH)

pat = re.compile(r"week(\d+)_judge(\d+)_score")
```

```

week_judge_cols = []
for c in df.columns:
    m = pat.match(c)
    if m:
        week_judge_cols.append((int(m.group(1)), int(m.group(2)), c))
week_judge_cols.sort(key=lambda x: (x[0], x[1]))
MAX_WEEK = max({w for w, j, c in week_judge_cols})

# =====
# 2) 工具函数：赛季矩阵、淘汰识别
# =====

def season_matrices(df_season: pd.DataFrame):
    contestants = df_season["celebrity_name"].tolist()
    N = len(contestants)
    T = MAX_WEEK

    J = np.full((T, N), np.nan, dtype=float)
    ran = np.zeros(T, dtype=bool)

    for t in range(1, T + 1):
        cols_t = [c for (w, j, c) in week_judge_cols if w == t]
        mat = df_season[cols_t].to_numpy(dtype=float)

        if np.all(np.isnan(mat)):  # 本周末播出
            ran[t - 1] = False
            continue

```

```

        ran[t - 1] = True

        J[t - 1, :] = np.nansum(mat, axis=1) # 忽略 NaN 求和

    return contestants, J, ran

def active_mask(J_t: np.ndarray):
    # 题面：淘汰后周次分数记 0；因此 >0 可视为在赛
    return np.isfinite(J_t) & (J_t > 0)

def elimination_set(J: np.ndarray, ran: np.ndarray):
    """
    t 周在赛、t+1 周变成 0 => t 周结束后淘汰
    返回 E[t]: 第 t 周结束淘汰者 index 集合
    """
    T, N = J.shape
    E = [set() for _ in range(T)]
    for t in range(T - 1):
        if (not ran[t]) or (not ran[t + 1]):
            continue

        act_t = active_mask(J[t])
        act_t1 = active_mask(J[t + 1])
        elim = np.where(act_t & (~act_t1))[0]
        E[t] = set(elim.tolist())

    return E

# =====
# 3) 问题 1 核心: Percent scheme 反推投票份额  $\hat{v}$ 
#     目标: 最大熵 + 平滑

```

```

# 约束：每周 sum(v)=1、非负、真实淘汰者合成得分最低
# =====

def solve_percent_inverse(contestants, J, ran, E, lam=2.0, eps_margin=1e-6, maxiter=2000):
    T, N = J.shape

    var_index = {}
    active_sets = {}
    idx = 0

    for t in range(T):
        if not ran[t]:
            continue

        act = np.where(active_mask(J[t]))[0]

        active_sets[t] = act

        for i in act:
            var_index[(t, i)] = idx
            idx += 1

    nvars = idx
    bounds = [(0.0, 1.0)] * nvars

    #  $q_{\{t,i\}} = J_{\{t,i\}} / \sum J_{\{t,*\}}$ 
    q = {}

    for t in range(T):
        if not ran[t]:
            continue

        act = active_sets[t]

        denom = np.nansum(J[t, act])

        if not np.isfinite(denom) or denom <= 0:

```

```

        continue

    for i in act:

         $q[(t, i)] = J[t, i] / \text{denom}$ 

def v_of(x, t, i):

    k = var_index.get((t, i), None)

    return x[k] if k is not None else 0.0

def objective(x):

    # minimize sum v log v + lam * sum (v_t - v_{t-1})^2

    ent = 0.0

    for (t, i), k in var_index.items():

        v = x[k]

        ent += v * math.log(max(v, 1e-12))

    smooth = 0.0

    prev_t = None

    for t in range(T):

        if not ran[t]:

            continue

        if prev_t is None:

            prev_t = t

            continue

        common = set(active_sets[prev_t].tolist()).intersection(active_sets[t].tolist())

        for i in common:

            smooth += (v_of(x, t, i) - v_of(x, prev_t, i)) ** 2

        prev_t = t

    return ent + lam * smooth

```



```

constraints = []

# 每周归一化: sum v = 1
for t in range(T):
    if not ran[t]:
        continue

    inds = [var_index[(t, i)] for i in active_sets[t]]

    constraints.append({
        "type": "eq",
        "fun": (lambda inds=inds: (lambda x: np.sum(x[inds]) - 1.0))()
    })

# 淘汰一致性: 真实淘汰者合成得分最低
#  $C_{\{t,i\}} = q_{\{t,i\}} + v_{\{t,i\}}$ 
for t in range(T):
    if (not ran[t]) or (len(E[t]) == 0):
        continue

    act = active_sets[t].tolist()

    survivors = [j for j in act if j not in E[t]]

    for e in list(E[t]):
        qe = q.get((t, e), 0.0)

        for j in survivors:
            qj = q.get((t, j), 0.0)

            constraints.append({
                "type": "ineq",
                "fun": (lambda t=t, e=e, j=j, qe=qe, qj=qj:
                    (lambda x: (qj + v_of(x, t, j)) - (qe + v_of(x, t, e)) -
                        eps_margin))()
            })

```

```

    ))

# 初始点：每周均匀
x0 = np.zeros(nvars)

for t in range(T):
    if not ran[t]:
        continue

    act = active_sets[t]

    val = 1.0 / len(act) if len(act) > 0 else 0.0

    for i in act:
        x0[var_index[(t, i)]] = val

res = minimize(
    objective,
    x0,
    method="SLSQP",
    bounds=bounds,
    constraints=constraints,
    options={"maxiter": maxiter, "ftol": 1e-9, "disp": False},
)

if not res.success:
    raise RuntimeError(f"Vote inverse solve failed: {res.message}")

V_hat = np.zeros((T, N), dtype=float)

for (t, i), k in var_index.items():
    V_hat[t, i] = res.x[k]

return V_hat, q, active_sets, res

```

```

# =====
# 4) 组装问题 3 建模数据集
# =====

def build_problem3_dataset(df, seasons_to_use, lam=2.0):

    rows = []

    for s in seasons_to_use:

        df_s = df[df["season"] == s].copy()

        if df_s.empty:

            continue

        contestants, J, ran = season_matrices(df_s)

        E = elimination_set(J, ran)

        # 反推投票份额

        V_hat, q_map, active_sets, res = solve_percent_inverse(

            contestants, J, ran, E, lam=lam

        )

        # 静态特征（每人一行）

        static_cols = [

            "celebrity_name",

            "ballroom_partner",

            "celebrity_industry",

            "celebrity_homestate",

            "celebrity_homecountry/region",

            "celebrity_age_during_season",

            "season",

```

```

        "placement",
    ]

    static =
df_s[static_cols].drop_duplicates(subset=["celebrity_name"]).set_index("celebrity_name")

# 逐周生成样本（只取在赛周）
for t in range(MAX_WEEK):
    if not ran[t]:
        continue

    act = active_sets.get(t, np.array([], dtype=int))

    if len(act) == 0:
        continue

    denom = np.nansum(J[t, act])

    if not np.isfinite(denom) or denom <= 0:
        continue

    for i in act:
        name = contestants[i]
        st = static.loc[name]

        J_it = J[t, i]

        if (not np.isfinite(J_it)) or (J_it <= 0):
            continue

        q_it = J_it / denom          # 评委份额
        v_it = V_hat[t, i]          # 观众份额（估计）

    rows.append({

```

```

        "season": int(s),
        "week": int(t + 1),
        "celebrity_name": name,
        "ballroom_partner": st["ballroom_partner"],
        "industry": st["celebrity_industry"],
        "homestate": st["celebrity_homestate"],
        "homecountry": st["celebrity_homecountry/region"],
        "age": float(st["celebrity_age_during_season"]),
        "placement": float(st["placement"]) if pd.notna(st["placement"]) else
np.nan,

        "judge_share": float(q_it),
        "fan_share": float(v_it),
    })

return pd.DataFrame(rows)

# =====
# 5) 特征处理 + 两通道模型 (Ridge)
# =====

def prepare_features(data: pd.DataFrame):
    d = data.copy()

    for col in ["industry", "homestate", "homecountry", "ballroom_partner"]:
        d[col] = d[col].fillna("Unknown").astype(str)

    d["age"] = d["age"].astype(float)
    d["week"] = d["week"].astype(int)

    X = pd.get_dummies(

```

```

        d[["age", "week", "industry", "homestate", "homecountry", "ballroom_partner"]],
        drop_first=True
    )
    return X, d

def fit_two_channel_ridge(X: pd.DataFrame, dprep: pd.DataFrame, alpha=1.0, test_size=0.25,
random_state=42):
    yJ = dprep["judge_share"].values

    eps = 1e-6
    v = np.clip(dprep["fan_share"].values, eps, 1 - eps)
    yV = np.log(v / (1 - v)) # logit

    X_train, X_test, yJ_train, yJ_test, yV_train, yV_test = train_test_split(
        X, yJ, yV, test_size=test_size, random_state=random_state
    )

    modelJ = Pipeline([("scaler", StandardScaler()), ("ridge", Ridge(alpha=alpha))])
    modelV = Pipeline([("scaler", StandardScaler()), ("ridge", Ridge(alpha=alpha))])

    modelJ.fit(X_train, yJ_train)
    modelV.fit(X_train, yV_train)

    predJ = modelJ.predict(X_test)
    predV = modelV.predict(X_test)

    metrics = {
        "Judge_R2": r2_score(yJ_test, predJ),
        "Judge_RMSE": float(np.sqrt(mean_squared_error(yJ_test, predJ))),
    }

```

```

        "FanLogit_R2": r2_score(yV_test, predV),

        "FanLogit_RMSE": float(np.sqrt(mean_squared_error(yV_test, predV))),

    }

    coefJ = modelJ.named_steps["ridge"].coef_
    coefV = modelV.named_steps["ridge"].coef_

    coef_df = pd.DataFrame({

        "feature": X.columns,

        "beta_judge": coefJ,

        "beta_fan": coefV,

    })

    coef_df["abs_judge"] = coef_df["beta_judge"].abs()

    coef_df["abs_fan"] = coef_df["beta_fan"].abs()

    return modelJ, modelV, metrics, coef_df, yJ, yV

# =====

# 6) 可视化

# =====

def plot_top_coefficients(coef_df, top_n=12):

    topJ = coef_df.sort_values("abs_judge", ascending=False).head(top_n).iloc[:,-1]

    plt.figure(figsize=(10, 6))

    plt.barh(topJ["feature"], topJ["beta_judge"])

    plt.title(f"Top {top_n} standardized coefficients — Judge channel (Ridge)")

    plt.xlabel("Coefficient")

    plt.grid(True, axis="x", alpha=0.3)

```

```

plt.tight_layout()

plt.show()

topV = coef_df.sort_values("abs_fan", ascending=False).head(top_n).iloc[::-1]

plt.figure(figsize=(10, 6))

plt.barh(topV["feature"], topV["beta_fan"])

plt.title(f"Top {top_n} standardized coefficients — Fan channel (logit vote share, Ridge)")

plt.xlabel("Coefficient")

plt.grid(True, axis="x", alpha=0.3)

plt.tight_layout()

plt.show()

```

```

def plot_coef_scatter(coef_df):

    plt.figure(figsize=(7, 7))

    plt.scatter(coef_df["beta_judge"], coef_df["beta_fan"])

    lim = max(coef_df["beta_judge"].abs().max(), coef_df["beta_fan"].abs().max())

    plt.plot([-lim, lim], [-lim, lim], linewidth=1)

    plt.xlabel("Judge-channel coefficient (standardized)")

    plt.ylabel("Fan-channel coefficient (standardized, logit)")

    plt.title("Feature effects: Judge vs Fan (Ridge coefficients)")

    plt.grid(True, alpha=0.3)

    plt.tight_layout()

    plt.show()

```

```

def extract_partner_effects(coef_df):

    mask = coef_df["feature"].str.startswith("ballroom_partner_")

    sub = coef_df[mask].copy()

    sub["partner"] = sub["feature"].str.replace("ballroom_partner_", "", regex=False)

    return sub

```



```
def plot_partner_effects(partner_df, channel="beta_fan", top_n=8, title_suffix="Fan"):
```

```
    sub = partner_df.sort_values(channel, ascending=False)
```

```
    top = pd.concat([sub.head(top_n), sub.tail(top_n)]).copy()
```

```
    top = top.sort_values(channel)
```

```
    plt.figure(figsize=(10, 6))
```

```
    plt.barh(top["partner"], top[channel])
```

```
    plt.title(f"Partner effects (Ridge) — {title_suffix} channel (top  $\pm$  {top_n})")
```

```
    plt.xlabel("Coefficient (standardized)")
```

```
    plt.grid(True, axis="x", alpha=0.3)
```

```
    plt.tight_layout()
```

```
    plt.show()
```

```
def plot_pred_vs_true(model, X, y, title):
```

```
    yhat = model.predict(X)
```

```
    plt.figure(figsize=(6, 6))
```

```
    plt.scatter(y, yhat)
```

```
    lim = [min(y.min(), yhat.min()), max(y.max(), yhat.max())]
```

```
    plt.plot(lim, lim, linewidth=1)
```

```
    plt.xlabel("True")
```

```
    plt.ylabel("Predicted")
```

```
    plt.title(title)
```

```
    plt.grid(True, alpha=0.3)
```

```
    plt.tight_layout()
```

```
    plt.show()
```

```
# =====
```

```

# 7) 主程序：先跑一个赛季（建议 27），再扩展

# =====

if __name__ == "__main__":
    # 建议先用 [27] 或 [2,4,11,27] 验证；批量跑 list(range(3,28)) 会慢
    seasons_to_use = [27]

    lam_for_vote_inverse = 2.0

    ridge_alpha = 1.0

    data3 = build_problem3_dataset(df, seasons_to_use, lam=lam_for_vote_inverse)

    print("Problem3 dataset:", data3.shape)

    print(data3.head())

    X, dprep = prepare_features(data3)

    modelJ, modelV, metrics, coef_df, yJ, yV = fit_two_channel_ridge(X, dprep,
alpha=ridge_alpha)

    print("\n=== Model metrics (holdout) ===")

    for k, v in metrics.items():
        print(f'{k}: {v:.4f}')

    # 1) 特征重要性图
    plot_top_coefficients(coef_df, top_n=12)

    # 2) Judge vs Fan 系数散点图
    plot_coef_scatter(coef_df)

    # 3) 舞者效应对比图
    partner_df = extract_partner_effects(coef_df)

```

```

plot_partner_effects(partner_df, channel="beta_judge", top_n=6, title_suffix="Judge")

plot_partner_effects(partner_df, channel="beta_fan", top_n=6, title_suffix="Fan (logit)")

# 4) 预测诊断图

plot_pred_vs_true(modelJ, X, yJ, "Judge channel: true vs predicted")

plot_pred_vs_true(modelV, X, yV, "Fan channel (logit): true vs predicted")

```

问题 4

问题 4 分析（新的投票制度设计）

4、设计一种你认为更“公平”或“更有观赏性”的评委与观众投票结合方式，并说明理由。这一问属于机制设计问题，重点不在于提出一个看起来新颖的规则，而在于说明该规则为何在结构上优于现有制度。一个合理的思路是让评委权重与观众投票的稳定性挂钩：当观众投票非常分散或极端时，提高评委权重以避免偶然性主导结果；当观众意见较为一致时，则更多尊重大众选择。这种设计体现的是“动态平衡”思想，即制度不是固定偏向某一方，而是根据当周数据结构自动调整。新制度的优越性需要通过模拟验证，例如比较不同制度下淘汰结果的稳定性、极端事件出现频率以及整体排名的一致性，从而证明新机制在公平性或观赏性上具有系统性改进，而不是个别案例上的巧合。

解题思路：

1. 设计目标：把“好制度”写成可优化的指标体系

设赛季 s 、周 t 的在赛集合为 $\mathcal{A}_{s,t}$ 。每位选手有两类核心输入：评委评分 $J_{s,i,t}$ 与观众投票份额 $v_{s,i,t}$ （可用问题 1 的估计 $\hat{v}_{s,i,t}$ 作为训练/评估用代理）。新制度的本质是定义一个合成函数 $F(\cdot)$ ，生成当周“淘汰风险/合成成绩”并据此淘汰。

为了系统设计，先提出可量化的制度目标函数（多目标）：

- **公平性 (Fairness)**：不因与舞蹈无关的特征（行业、州、年龄、搭档舞者等）而出现系统性偏置；
- **稳定性 (Stability)**：对小扰动（评分误差、投票波动）不敏感，减少“爆冷”；
- **观众代表性 (Representation)**：观众偏好能产生实质影响，但不至于压倒专业评价；
- **抗操纵性 (Robustness)**：防止“极端粉丝团集中投票”让低表现者长期存活；
- **可解释性 (Interpretability)**：规则简单透明，观众与选手可理解；
- **争议鲁棒性 (Controversy robustness)**：对历史争议季（如 27 季）能避免不符合直觉的路径。

把这些目标写成指标：例如对稳定性，可以定义当周淘汰边际 $m_{s,t} = \min_{j \in E_{s,t}} F_{s,j,t} - \max_{e \in E_{s,t}} F_{s,e,t}$ （边际越大越稳定）；对公平性，可以定义“敏感特征影响”在制度淘汰概率中的系数幅度或差异（见第 6 节）；对抗操纵性，可定义“低评委分但高投票者的存活概率”受限（见第 4 节约束）。

2. 输入标准化：先把评委与投票变成同尺度、可比的量

不同赛季评委人数不同、分数尺度不同（含奖励分、平均等），直接加总会产生尺度偏差。

推荐使用每周占比：

评委占比 $q_{s,i,t} = \frac{J_{s,i,t}}{\sum_{j \in \mathcal{A}_{s,t}} J_{s,j,t}}$ ，满足 $\sum_{i \in \mathcal{A}_{s,t}} q_{s,i,t} = 1$ 。

观众占比 $v_{s,i,t}$ 也满足 $\sum_{i \in \mathcal{A}_{s,t}} v_{s,i,t} = 1$ （若用问题 1 估计则取 $\hat{v}_{s,i,t}$ ）。

为了降低极端值影响，可对两者做“温和压缩”变换（尤其是投票可能更极端）：

- 对评委： $q'_{s,i,t} = \frac{q_{s,i,t}^\eta}{\sum_j q_{s,j,t}^\eta}$ ， $\eta \in (0,1]$ ；
- 对投票： $v'_{s,i,t} = \frac{v_{s,i,t}^\gamma}{\sum_j v_{s,j,t}^\gamma}$ ， $\gamma \in (0,1]$ 。

当 $\gamma < 1$ 时，投票分布被“拉平”，可抑制粉丝团极端集中投票的冲击；当 $\eta < 1$ 时也能减小评委极端分差对淘汰的瞬时支配。

3. 新制度核心：一个“自适应权重 + 稳健惩罚”的合成得分

现行两类制度（Percent/Rank）本质上是固定方式合成。新制度建议采用一个更稳健且可解释的合成得分：

$$S_{s,i,t} = \alpha_t q'_{s,i,t} + (1 - \alpha_t) v'_{s,i,t} - \lambda \cdot \text{RiskPenalty}_{s,i,t},$$

其中：

- $\alpha_t \in [0,1]$ 是随周次变化的权重：早期更依赖评委（鼓励技术、避免早期爆冷），后期逐步增加观众权重（体现参与感与人气）。
- $\text{RiskPenalty}_{s,i,t}$ 是对“明显技术弱者靠投票苟活”的稳健惩罚项；
- $\lambda \geq 0$ 控制惩罚强度。

淘汰规则：每周淘汰合成得分最低者（或最低 k_t 人）：

$$E^{\text{new}}_{s,t} = \text{Bottom-}k_t(S_{s,i,t} | i \in \mathcal{A}_{s,t}).$$

4. 稳健惩罚项设计：限制“低评委高投票”长期幸存（抗操纵）

惩罚项的目标是：如果某人评委占比长期处于尾部，但投票占比很高，则制度应降低其存活概率，从而减少争议。

一种简单透明的设计是基于“评委相对分位数”的惩罚：

先定义评委分位数（越大越好）：

$$p^{(j)}_{s,i,t} = \frac{\text{RankDesc}(J_{s,i,t}) - 1}{|\mathcal{A}_{s,t}| - 1}, \text{ 则 } p^{(j)} \in [0,1].$$

定义“低评委阈值” τ_j （例如 0.2 表示处于后 20%）。惩罚项可设为：

$$\text{RiskPenalty}_{s,i,t} = \max(0, \tau_j - p^{(j)}_{s,i,t}).$$

直观解释：当选手评委表现进入后段时，合成得分会被扣减，扣减幅度随其落后程度增加。也可以用连续化的 logistic 惩罚，使得惩罚更平滑：

$$\text{RiskPenalty}_{s,i,t} = \sigma\left(\frac{\tau_J - p^{(J)}_{s,i,t}}{\kappa}\right), \text{ 其中 } \sigma(x) = \frac{1}{1 + \exp(-x)}, \kappa > 0 \text{ 控制平滑程}$$

度。

如果你想更“机制化”，可让惩罚基于“评委份额与投票份额的不一致程度”：

$$\text{RiskPenalty}_{s,i,t} = \max(0, v'_{s,i,t} - q'_{s,i,t} - \delta), \text{ 其中 } \delta \geq 0 \text{ 允许一定范围内的“观众偏爱”不被惩罚。}$$

5. 自适应权重 α_t ：随周次从“技术导向”过渡到“人气导向”

固定 α 的问题是：早期观众信息少，易被名气/粉丝量主导；后期舞蹈水平差距收敛，观众参与更重要。因此建议 α_t 随周次变化：

一种线性方案：

$$\alpha_t = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \left(1 - \frac{t-1}{T_s-1}\right),$$

其中 $\alpha_{\max}$ 是早期评委权重（如 0.7）， $\alpha_{\min}$ 是后期评委权重（如 0.4）。

一种更平滑的 logistic 方案：

$$\alpha_t = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \cdot \frac{1}{1 + \exp(\beta(t - t_0))},$$

其中 t_0 是拐点周次， $\beta > 0$ 控制过渡速度。

这样设计的可解释性很强：节目越到后期越接近“全民投票”，但不会在早期被粉丝团瞬间带走。

6. 公平性约束：用问题 3 的“特征影响”来做制度校正

问题 3 建模得到：某些非舞蹈因素（行业、年龄、州、舞者搭档）对观众投票或评委打分有系统影响。新制度可以把这些影响作为“偏置项”进行校正，避免制度强化偏见。

设从问题 3 的观众通道模型得到特征对投票的预测 $\hat{v}^{\text{bias}}_{s,i,t}$ （仅由静态特征预测的人气倾向），例如：

$$\text{logit}(\hat{v}_{s,i,t}) = x_{s,i}^\top \beta^{(V)} + \epsilon,$$

则可定义“去偏投票”：

$$\tilde{v}_{s,i,t} \propto \frac{v_{s,i,t}}{\exp(x_{s,i}^\top \beta^{(V)})},$$

再归一化使得 $\sum_i \tilde{v}_{s,i,t} = 1$ 。

同理也可对评委做去偏（若存在系统性偏差）：

$$\tilde{q}_{s,i,t} \propto \frac{q_{s,i,t}}{\exp(x_{s,i}^\top \beta^{(J)})}。$$

然后在新制度中用 $\tilde{v}$ 、 $\tilde{q}$ 代替原始 v 、 q 。这相当于引入“公平性校正”，强调舞蹈表现而非背景特征。

7. 二阶段机制融合：保留节目戏剧性（Bottom-2 + Judges Save）

赛题提到后期制度有“倒数两名后评委二选一淘汰”。新制度可以将其作为“安全阀”保留，同时用更合理的第一阶段筛选倒数两名：

第一阶段：用新制度得分确定倒数两名：

$$B^{\text{new}}_{s,t} = \text{Bottom2}(S_{s,i}, t | i \in \mathcal{A}_{s,t})。$$

第二阶段：评委在倒数两名中做选择。可以用确定性或概率模型：

$$P(a | a, b) = \frac{1}{1 + \exp(\gamma(J_{s,a,t} - J_{s,b,t}))}。$$

这样既保留节目“现场救人”的戏剧性，也能让倒数两名的筛选更稳健、更少争议。

8. 参数选择：把制度设计变成“可优化”的问题（用历史赛季训练）

新制度包含参数 $\theta = (\alpha_{\min}, \alpha_{\max}, \beta, t_0, \gamma, \eta, \tau_J, \lambda, \delta, \kappa, \dots)$ 。你们可以把参数通过历史赛季进行“离线训练/调参”，目标是综合优化多指标：

定义总损失：

$$\mathcal{L}(\theta) = w_1 \mathcal{L}_{\text{consistency}}(\theta) + w_2 \mathcal{L}_{\text{stability}}(\theta) + w_3 \mathcal{L}_{\text{fairness}}(\theta) + w_4 \mathcal{L}_{\text{controversy}}(\theta)。$$

其中：

- $\mathcal{L}_{\text{consistency}}$ ：新制度在历史数据上不应产生过多“离谱淘汰”，可用与原制度淘汰差异率或预测准确率度量；
- $\mathcal{L}_{\text{stability}} = -\sum s, t m_{s,t}$ （边际越大越好，所以取负）；
- $\mathcal{L}_{\text{fairness}}$ ：对敏感特征的影响惩罚，例如若用生存模型得到敏感特征系数向量 ϕ ，可设 $\mathcal{L}_{\text{fairness}} = \|\phi\|_2^2$ ；
- $\mathcal{L}_{\text{controversy}}$ ：对争议赛季设置额外惩罚，比如要求“低评委长期存活”的次数不超过阈值。

最终求解 $\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$ 。实现上可采用网格搜索或贝叶斯优化，论文里写“我们用历史赛季交叉验证选择参数”即可。

10. 一句话总结新制度（写在摘要里很好用）

你们可以把新制度概括为：

“我们提出一种 $S_{s,i,t} = \alpha_i q'_{s,i,t} + (1 - \alpha_i) v'_{s,i,t} - \lambda \cdot \text{RiskPenalty}_{s,i,t}$ 的自适应加权制

度，其中 α_t 随周次从评委导向过渡到观众导向，并对‘低评委高投票’情形施加稳健惩罚，从而在历史赛季上同时提升稳定性、降低背景特征偏置并减少争议性淘汰。”

Python 代码: